

Databases, SQL, Indexes, and Transactions

A database is not just a place to store JSON. Its schema and transaction boundaries encode invariants, while indexes trade read efficiency for write and storage cost. Learn these decisions using a small order or reservation model and inspect actual query behavior.

For: Backend and data-oriented engineers who can write simple queries but need to reason about data grain, constraints, indexing, isolation, and safe concurrent updates.

Suggested pace: 4 weeks with a local relational database | Basic programming and SQL

What you will be able to do

- Define table grain, keys, constraints, and relationships before writing joins.
- Use indexes based on predicates and access patterns rather than intuition alone.
- Explain transaction isolation and concurrency failures with concrete examples.
- Handle migrations, retries, and backups as part of the data lifecycle.

Build the right mental model

Grain prevents double counting

Know what one row represents in every table and intermediate result. Joining multiple one-to-many relationships can multiply rows. Aggregate at the correct key before combining measures, and use constraints to express identities that must be unique.

Indexes support workloads

An index can reduce lookup and ordering work when it matches the query, but it consumes space and must be maintained on writes. Inspect plans and representative row counts. A sequential scan can be correct and efficient for a small table or a broad query.

Transactions protect boundaries

Atomicity groups changes, while isolation defines what concurrent operations can observe. A read followed by a write does not automatically protect an invariant against another transaction. Constraints, conditional updates, locking, or stronger isolation may be necessary.

Study with evidence.

Use PostgreSQL or another well-documented local database and synthetic data. Write the expected rows before running a query. For concurrency exercises, use two sessions and explicit steps instead of relying on chance timing.

Week 1: Model and query

Build customers, orders, and items with primary and foreign keys. Practice joins, grouping, NULL behavior, and window functions. Test empty groups, duplicates, and ties rather than only clean sample data.

Deliverable and check: A schema diagram and queries whose output grain and edge cases are documented.

Week 2: Index and measure

Create a larger synthetic dataset and inspect query plans. Compare selective lookups, ordered history, and broad scans. Add a composite index for a real access pattern and measure read and write effects.

Deliverable and check: A before/after plan with workload size, query, and limitations; no unsupported claim that every index makes queries faster.

Week 3: Concurrent correctness

Use two sessions to reproduce a lost update or last-item reservation race. Add the database-enforced invariant and choose an appropriate transaction strategy. Test retry behavior after a conflict or ambiguous client timeout.

Deliverable and check: A deterministic concurrency test that never creates two owners for the same exclusive resource.

Week 4: Operational data changes

Practice a backward-compatible schema change, migration rollback or forward repair, backup, and restore in a disposable database. Explain which application versions can coexist during deployment.

Deliverable and check: A migration runbook and a tested restore, not merely a backup file whose recovery has never been attempted.

A repeatable practice session

Spend about 15 minutes recalling the previous topic without notes, 45 minutes on today's problem or lab, and 15 minutes explaining and checking the result. Record the failed assumption, not just the final answer. Revisit weak material after a few days and again the following week.

Apply it. Explain it.

Reserve the final seat safely

Two clients try to reserve the last seat for an event. Both first read that the seat is available. The application must allow at most one active reservation, including when requests are retried.

A worked approach

1. Model a stable seat/event identity and enforce a unique active ownership rule in the database. Define reservation states and whether an expired hold can be reclaimed. An application precheck alone is not the invariant boundary.
2. Perform the claim using an appropriate transaction or conditional update that succeeds only for the expected available state. Return a clear conflict when another transaction wins instead of silently overwriting ownership.
3. Run two sessions through the same interleaving, then retry the winning request with its original request ID. Verify one reservation and one logical result, and distinguish a known conflict from a timeout with an unknown outcome.

Expected result

Correctness comes from the database boundary and request identity, not from the timing of the availability check. The loser receives a defined result and retries do not create extra reservations.

Verify the result

- Can two concurrent claims violate the unique ownership rule?
- Does retrying the same logical request preserve its result?
- Are lock duration and transaction failure handled explicitly?

Extend the exercise

Add expiring holds and a payment callback. Describe what happens when payment succeeds after expiry; do not automatically confirm a seat that another customer now owns.

Make the learning stick.

Why not put an index on every column?

Indexes add write work, storage, and maintenance cost. Their usefulness depends on predicates, selectivity, ordering, and query shape. Choose them from measured workloads and inspect plans rather than indexing columns indiscriminately.

Does ACID mean every transaction is serializable?

No. Isolation levels differ, and a database's default may allow anomalies relevant to your invariant. Read the specific engine's contract and test the concurrency pattern you need to protect.

Why test restore if backups succeed?

A backup command can succeed while recovery assumptions, permissions, versions, or missing dependencies make restoration fail. A recovery plan is credible only after a representative restore and validation of the recovered data.

Avoid these traps

- Joining child tables without checking row multiplication.
- Using application checks instead of database-enforced invariants.
- Claiming a backup strategy without a tested recovery path.

Ready for the next step?

- Schema keys and output grain are explicit.
- Index choices are linked to observed access patterns.
- Concurrent updates preserve the required invariant.
- Migrations and restore procedures have been exercised.

Reliable next reads

- [PostgreSQL tutorial](#)
- [PostgreSQL transaction isolation](#)
- [PostgreSQL indexes](#)

An independent preparation guide, not an employer-endorsed syllabus or a guarantee of an offer. Company examples are practice contexts, not verified past interview questions. Use only public, synthetic, or authorized material in exercises.